

Module 1: Describing the Data

Descriptive Statistics in Simple Steps

Dwaipayan Roy

Department of Computational and Data Sciences



24th September, 2026

What We Will Do in This Module

- 1 Open the data file
- 2 Look at the first few rows
- 3 See how many rows and columns there are
- 4 Count households in each group
- 5 Calculate averages (mean and median)
- 6 Compare groups (rural vs urban, men vs women)
- 7 Check for missing values

All of this is called **descriptive statistics** - describing the data before we try to test anything.

What Is a Dataset?

- A dataset is a table of information
- Each **row** is one household
- Each **column** is one piece of information
- Our dataset has **200 households** and **10 columns**

Think of it as a spreadsheet. Rows are households. Columns are things we know about each household.

What Is in Our Dataset?

Numbers

- age - age of household head
- education_years - years of schooling
- household_size - number of members
- income - monthly income in rupees
- wellbeing_score - 0 to 100

Categories

- region_type - Rural or Urban
- gender - Male or Female
- land_ownership - Yes or No
- shg_membership - Yes or No

Step 1: Loading the Tools

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
print("Tools ready!")
```

Step 1: Loading the Tools

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
print("Tools ready!")
```

`import` means "bring in a tool". Each tool does a different job.

Step 1: Loading the Tools

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
print("Tools ready!")
```

`import` means "bring in a tool". Each tool does a different job.

pandas - works with tables

numpy - does maths

matplotlib - makes charts

seaborn - makes prettier charts

scipy - runs statistical tests

Expected output

Tools ready!

Step 2: Opening the Data File

```
df = pd.read_csv("village_survey.csv")
```

We read the CSV file and store it in a variable called `df`. The name `df` is short for "data frame" - a fancy word for a table.

Step 2: Opening the Data File

```
df = pd.read_csv("village_survey.csv")
```

We read the CSV file and store it in a variable called `df`. The name `df` is short for "data frame" - a fancy word for a table.

- `pd.read_csv()` - opens CSV files
- `"village_survey.csv"` - file name
- `df` - name we give the table

Step 2: Opening the Data File

```
df = pd.read_csv("village_survey.csv")
```

We read the CSV file and store it in a variable called `df`. The name `df` is short for "data frame" - a fancy word for a table.

- `pd.read_csv()` - opens CSV files
 - `"village_survey.csv"` - file name
 - `df` - name we give the table
- `df = pd.read_csv("village_survey.tsv", sep="\t")`

Step 2: Opening the Data File

```
df = pd.read_csv("village_survey.csv")
```

We read the CSV file and store it in a variable called `df`. The name `df` is short for "data frame" - a fancy word for a table.

- `pd.read_csv()` - opens CSV files
- `"village_survey.csv"` - file name
- `df` - name we give the table
- `df = pd.read_csv("village_survey.tsv", sep="\t")`
- Excel files (`.xlsx`, `.xls`):
`df = pd.read_excel("village_survey.xlsx", sheet_name="Sheet1")`

Step 2: Opening the Data File

```
df = pd.read_csv("village_survey.csv")
```

We read the CSV file and store it in a variable called `df`. The name `df` is short for "data frame" - a fancy word for a table.

- `pd.read_csv()` - opens CSV files
- `"village_survey.csv"` - file name
- `df` - name we give the table
- `df = pd.read_csv("village_survey.tsv", sep="\t")`
- Excel files (`.xlsx`, `.xls`):
`df = pd.read_excel("village_survey.xlsx", sheet_name="Sheet1")`
- JSON:
`df = pd.read_json("village_survey.json")`

Step 3: Looking at the First Few Rows

```
print(df.head())
```

`head()` shows the first five rows. It is a quick way to check that the file opened properly.

	household_id	region_type	gender	age	education_years	...
0	HH1000	Rural	Male	48	2	...
1	HH1001	Urban	Female	54	7	...
2	HH1002	Rural	Male	45	9	...
3	HH1003	Rural	Male	51	8	...
4	HH1004	Urban	Male	49	13	...

Step 4: How Big Is the Dataset?

```
print(df.shape)
```

```
(200, 10)
```

shape tells us the size of the table. The first number is rows, the second is columns.

- **200** rows - 200 households
- **10** columns - 10 pieces of information per household

Step 5: What Are the Column Names?

```
print(df.columns.tolist())
```

```
['household_id', 'region_type', 'gender', 'age',  
 'education_years', 'household_size', 'income',  
 'land_ownership', 'shg_membership', 'wellbeing_score']
```

`columns.tolist()` gives us a simple list of all the column names. Useful when you forget a name.

Step 6: What Type Is Each Column?

```
print(df.dtypes)
```

```
household_id      object
region_type       object
gender            object
age               int64
education_years   int64
household_size    int64
income            int64
land_ownership    object
shg_membership     object
wellbeing_score   float64
dtype: object
```

object = text. int64 = whole number. float64 = number with decimals.

Step 7: How Many Households in Each Group?

```
print(df['region_type'].value_counts())
```

```
Rural      129  
Urban       71  
Name: count, dtype: int64
```

`value_counts()` counts how many rows fall into each category.

Step 7b: Counting Other Groups

```
print(df['gender'].value_counts())  
print(df['land_ownership'].value_counts())  
print(df['shg_membership'].value_counts())
```

Male	154
Female	46

No	111
Yes	89

No	137
Yes	63

The same command works for every categorical column. Just change the column name inside the square brackets.

Step 8: Showing Percentages

```
print(df['region_type'].value_counts(normalize=True) * 100)
```

```
Rural      65.0  
Urban      35.0  
Name: region_type, dtype: float64
```

Adding `normalize=True` turns counts into proportions. Multiplying by 100 gives percentages.

Step 9: The Average (Mean)

```
print(df['age'].mean())  
print(df['income'].mean())  
print(df['wellbeing_score'].mean())
```

```
47.3  
35148.29  
57.46
```

The **mean** is the average. Add up all the values and divide by the number of households.

Step 10: The Middle Value (Median)

```
print(df['age'].median())  
print(df['income'].median())
```

```
48.0  
26594.00
```

The **median** is the middle value. Half the households are above it, half below.

Step 11: Why Mean and Median Differ

Mean income: 34,216

Median income: 28,500

The mean is higher because a few very rich households pull it up.

When mean is much higher than median, the data is **skewed** - most households are below the average.

- For income, the **median** is usually more honest
- Report both if you can
- If mean and median are close, the data is symmetric

Step 12: Smallest and Largest Values

```
print(df['income'].min())  
print(df['income'].max())
```

```
4771.0  
164486.0
```

`min()` and `max()` show the smallest and largest values. These help us spot outliers - unusual values worth a second look.

Step 13: How Spread Out Is the Data?

```
print(df['income'].std())  
print(df['age'].std())
```

```
25209.70325941131  
11.4245075105799
```

The **standard deviation** tells us how spread out the values are. A small number means the values are close together; a large number means they are spread apart.

Step 14: All Summary Numbers at Once

```
print(df[['age', 'income', 'wellbeing_score']].describe())
```

`describe()` gives you all the key numbers in one table: count, mean, standard deviation, minimum, quartiles, and maximum.

Step 15: Comparing Rural and Urban

```
print(df.groupby('region_type')['income'].mean())
```

```
region_type
Rural      29219.896000
Urban      46208.731343
Name: income, dtype: float64
```

groupby() splits the data into groups, then calculates something within each group.

Step 15b: More Group Comparisons

```
print(df.groupby('region_type')['education_years'].mean())  
print(df.groupby('gender')['income'].mean())  
print(df.groupby('shg_membership')['wellbeing_score'].mean())
```

Rural	6.38
Urban	10.23
Male	30712.45
Female	36467.05
No	56.38
Yes	59.72

The same pattern works for any group and any numeric column. Just change the two names.

Step 16: Two Categories at Once

```
print(pd.crosstab(df['region_type'], df['land_ownership']))
```

land_ownership	No	Yes
region_type		
Rural	66	63
Urban	45	26

crosstab counts how many households fall into each combination of two categories.

Step 17: Are Any Values Missing?

```
print(df.isnull().sum())
```

```
household_id      0
region_type       0
gender            0
age              0
education_years   0
household_size    0
income            8
land_ownership    0
shg_membership    0
wellbeing_score   6
dtype: int64
```

`isnull()` checks for missing values. `sum()` adds them up. Real surveys always have some missing answers.

What Did We Learn in Module 1?

- 1 Open a CSV file with `pd.read_csv()`
- 2 Look at the first rows with `head()`
- 3 Check size with `shape`
- 4 Count categories with `value_counts()`
- 5 Calculate averages with `mean()` and `median()`
- 6 Compare groups with `groupby()`
- 7 Count combinations with `crosstab()`
- 8 Check missing values with `isnull().sum()`

A Note on Reading Numbers Honestly

- Always report the **median** alongside the mean for income
- Always check **how many values are missing**
- Never compare groups without checking their **sample sizes**
- A large difference between two small groups can be misleading

A Note on Reading Numbers Honestly

- Always report the **median** alongside the mean for income
- Always check **how many values are missing**
- Never compare groups without checking their **sample sizes**
- A large difference between two small groups can be misleading

Descriptive statistics are not neutral. What you choose to report shapes how readers understand the data.

Key Takeaways

- ① A dataset is a table: rows are households, columns are variables
- ② Numbers can be averaged; categories can be counted
- ③ Mean and median can differ - both matter
- ④ `groupby()` is the workhorse for comparing groups
- ⑤ Always check for missing values before analysing
- ⑥ Descriptive statistics are the foundation of every later test

Key Takeaways

- ① A dataset is a table: rows are households, columns are variables
- ② Numbers can be averaged; categories can be counted
- ③ Mean and median can differ - both matter
- ④ `groupby()` is the workhorse for comparing groups
- ⑤ Always check for missing values before analysing
- ⑥ Descriptive statistics are the foundation of every later test

Next: are these differences real or chance?